# What a Variable Actually Holds

*CSCI-121 · Elements of Computer Programming II*

Week 2, Day 1 · Tuesday, September 1 · SC 333 · 3:00–4:15

# What you leave with today

- What the **compiler** does for you that Python's interpreter does not
- The **box-and-arrow** picture — for a primitive, and for a reference
- Why == on two Strings is a question about **objects**, not text
- What is on **Q1** — Thursday, first ten minutes

> **This is the first syntax of the semester.** Week 1 had none on purpose. Handout out, pen in hand — you predict before we run anything.

# Predict first

*write it down before it goes on the screen*

# Who says `true`?

```
String a = "hi";
String b = new String("hi");
System.out.println(a == b);
```

> **Hands up.**

# Two rooms in one room

| | Recitation 2 | What you bring today |
|---|---|---|
| **R3** | Monday — done | You have already been bitten by every trap in this lecture |
| **R4** | Wednesday — not yet | You see it here first, and hit it tomorrow |

**R3:** you watched == return `false` on two strings that looked identical. Tell them what happened. Then I will tell you why — which is what a lecture is for and a lab period is not.

# Q1 — Thursday, first ten minutes

- Ten minutes · start of class · **on paper** · no notes
- Data types, operators, strings
- Everything on it comes from **today, Thursday, and your section this week**

> **Finish Recitation 2 with green tests and you have studied for it.** There is nothing else to revise.

# Static types

*the compiler is working for you, not against you*

# Java will not lose your data quietly

```
int x = 3.5;
            ← does not compile
```

```
int x = (int) 3.5;
            ← 3. You said so, and you accepted the loss
```

> Python would take the first line. Java refuses to throw away the `.5` unless **you** say so, out loud, in the code.

# Java widens for free.

Java **never** narrows without a cast.

**This is one of the two rules to write down today.**

# Which way is free?

| | Free? |
|---|---|
| `int` → `double` | **automatic** — nothing is lost |
| `double` → `int` | needs `(int)` — something *is* lost |
| `char` → `int` | **automatic** |
| `int` → `char` | needs `(char)` |

> The rule is not about which type is bigger. It is about whether **the value survives the trip**.

# A variable keeps its type

```
String name = "Ada";
name = 42;
            ← does not compile
```

In Python a name is a **luggage tag** — you can move it to any bag.

In Java the variable has a type and it keeps it. You give up flexibility. What you get back is that the compiler catches an entire category of bug **before your program ever runs**.

# Every error you hit this month is an error Python would have let you ship.

**Say that to yourself the next time `javac` shouts at you.**

# What a variable actually holds

*copy this drawing — everything else today is a consequence of it*

# Two kinds of variable

PRIMITIVE

```
int n = 5;
```


the box holds **the value itself**

REFERENCE

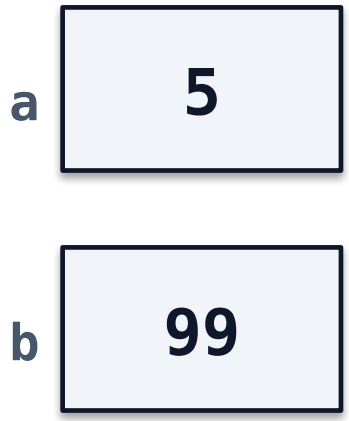
```
String s = "hi";
```


the box holds **an arrow**. The object lives elsewhere

You already have this. Recitation 0 gave you **stack** and **heap**. The primitive lives on the stack. The object lives on the heap — and what is on the stack is the arrow.

# Copy a primitive — you copy the value

```
int a = 5;
int b = a;
b = 99;
System.out.println(a);      // 5
```

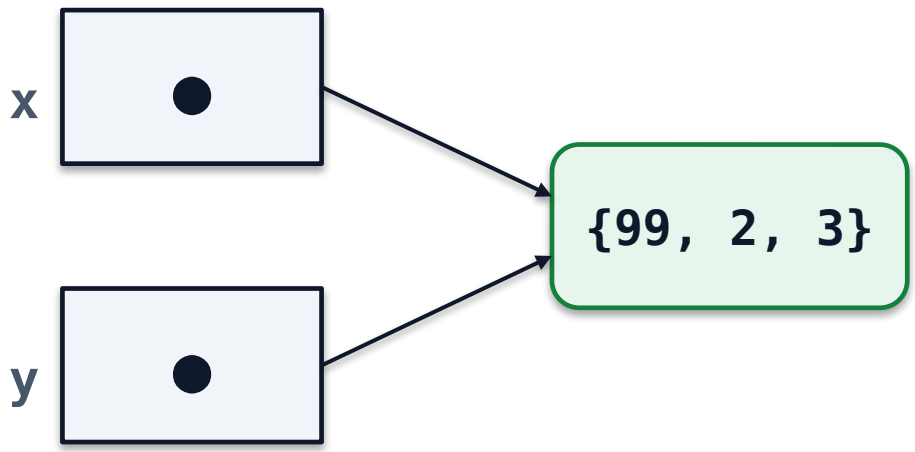



two boxes · nothing connects them

`b` got a **copy of the number**. Changing `b` cannot reach `a`, because there is nothing between them to travel along.

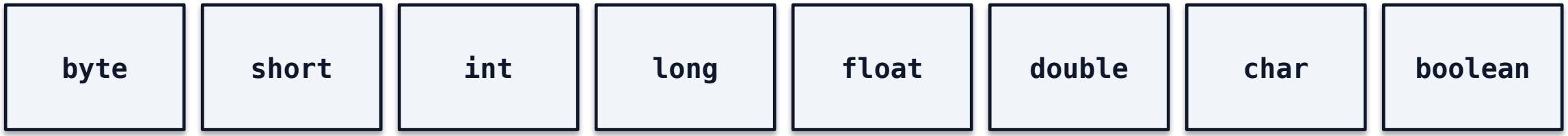
# Copy a reference — you copy the arrow

```
int[] x = {1, 2, 3};
int[] y = x;
y[0] = 99;
System.out.println(x[0]);   // 99
```




**One array. Two names for it.** Nothing was copied except the arrow — and this is the picture behind every surprise you will hit this month.

# Eight primitives. Everything else is a reference.




> Lowercase, always. **If the type starts with a capital letter it is a class**, and a variable of that type holds an arrow.

`String`, `int[]`, `Scanner`, `RobotController`, every class you write — all of them are the right-hand box on the last slide.

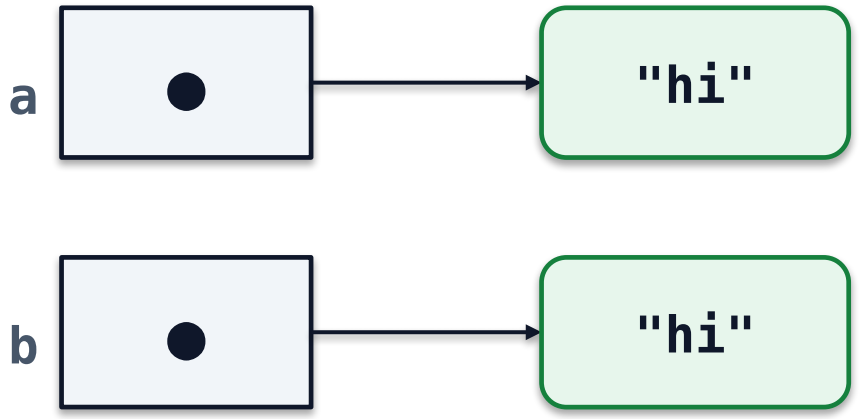
# `==` versus `.equals()`

*now collect the payoff*

# Back to the first prediction

```
String a = "hi";
String b = new String("hi");

a == b        // false
```




two objects · same letters

== compares **what is in the box**. The box holds an arrow. Two different arrows, pointing at two different objects that happen to contain the same letters.

# Two different questions

| | asks |
|---|---|
| `==` | are these **the same object**? |
| `.equals()` | do these have **the same contents**? |

> For primitives there is no arrow, so == compares values and does the obvious thing. `5 == 5` is `true` and always will be. **The trap only exists for objects.**

# The cruel part

```
String a = "hi";
String b = "hi";
System.out.println(a == b);   // true
```

Both are **literals**. Java stores identical literals once and points both names at the same object. So now it is `true`.

> `==` **appears to work** — and it will keep appearing to work for exactly as long as you test it with strings you typed yourself.

# No crash. No error. Just wrong.

The first time a string arrives from a `Scanner`, a file, or the network, it is a different object — and your program silently starts giving the wrong answer.

**That is why you met this in a lab before I said it in a lecture.**

# == for primitives.

`.equals()` for everything else.

**Every time. This is the second rule to write down.**

# `TimeDilation.java` — today, in one file

- `final double speedOfLight = 2.99792458e8;` — `final`, and scientific notation
- Every variable **declared with a type before it is used**
- `Math.sqrt` returns a `double`, and the whole expression stays `double`
- `printf("%1$.2f")` — formatting. That is Thursday, and exercise 7 of Recitation 2

> Run it with `0.9` and `10`. **Ten years for them, twenty-three for you** — that is `double` arithmetic doing something you would not want an `int` doing.

# Exit ticket — hand it in on your way out

- `int x = 7 / 2;` — what is x?
- Draw the box for `int n = 5;` and the box for `String s = "hi";`
- Two strings print the same text. == says `false`. **In one sentence, why?**

Tear-off page at the back of your handout. **Four minutes.** Name and section on it — I read these tonight.

# Before you go

- **Q1 — Thursday**, first ten minutes, on paper, no notes
- **Recitation 2** — finish at home, push, open the PR — **Sun Sep 6, 11:59 PM**
- **Drills** — Asterisks · Show me the Numbers — **Sun Sep 6**
- **Skill Builder 1** — **Sun Sep 13**. Exercises 6–8 of Recitation 2 are its spine
- Reading — Think Java **Ch 2**, **Ch 3**, **§6.10**, **§9.3**
- **R4:** your section is Wednesday — repo is `DSU-CSCI-121-F26/recitation2`
- **Everything I typed today** — fork it, break it: `DSU-CSCI-121-F26/lecture-week02-day1`

**Thursday: strings, casting, and what `javac` actually produces** — including a file that fails to compile, and therefore does not exist at all.

# == for primitives. `.equals()` for everything else.

*the box holds an arrow — that is the whole week*